

COMP 3311 DATABASE MANAGEMENT SYSTEMS

FINAL REVIEW EXERCISES

QUESTION 1

The **submitters** collection represents people who submit research proposals for review and the proposals they have submitted. The **reviewers** collection represents people who review submitted proposals and their reviews for the proposals to which they have been assigned.

submitters collection

```
{sid: , name: , email: ,  
  proposals: [{pid: , title: , area: }, ...]} // an array of proposal documents
```

reviewers collection

```
{rid: , name:, email:, expertise: ,  
  reviews: [{pid: , score: }, ...]} // an array of review documents
```

- a) Construct a MQL query to find the submitter name and the proposal title of proposals in the Database area. Use only the **find** method.

```
db.submitters.find({'proposals.area': 'Database'},  
  {'_id: 0, name: 1, 'proposals.title': 1});
```

submitters collection

{sid: , name: , email: ,
proposals: [{pid: , title: , area: }, ...]}

reviewers collection

{rid: , name: , email: , expertise: ,
reviews: [{pid: , score: }, ...]}

QUESTION 1 (CONT'D)

- b) Construct a MQL query to find the name, maximum and minimum score of each reviewer who reviewed exactly five proposals

```
db.reviewers.aggregate([
  {$unwind: "$reviews"},
  {$group: {
    _id: "$rid",
    name: {$first: "$name"},
    numReviews: {$count: {}},
    maxScore: {$max: "$reviews.score"},
    minScore: {$min: "$reviews.score"},
  }},
  {$match: {numReviews: {$eq: 5}}},
  {$project: {_id: 0, name: 1, maxScore: 1, minScore: 1}}
])
```

// group by the reviewer id
// keep the reviewer name
// count the number of reviews
// calculate the max and min scores
// keep score only if 5 proposals were reviewed

submitters collection

{sid: , name: , email: ,
proposals: [{pid: , title: , area: }, ...]}

reviewers collection

{rid: , name: , email: , expertise: ,
reviews: [{pid: , score: }, ...]}

QUESTION 1 (CONT'D)

- b) Construct a MQL query to find the name, maximum and minimum score of each reviewer who reviewed exactly five proposals

```
db.reviewers.aggregate([  
  {  
    $project: {  
      _id: 0, name: 1,  
      max: {  
        $max: '$reviews.score',  
      },  
      min: {  
        $min: '$reviews.score',  
      },  
      count: {  
        $size: '$reviews',  
      },  
    },  
    $match: {count: 5}  
  }  
]);
```

```
db.reviewers.aggregate([  
  {  
    $project: {  
      _id: 0, name: 1,  
      max: {  
        $max: '$reviews.score',  
      },  
      min: {  
        $min: '$reviews.score',  
      },  
      count: {  
        $cond: {  
          if: {  
            $isArray: '$reviews',  
          },  
          then: {  
            $size: '$reviews',  
          },  
          else: 0  
        },  
      },  
    },  
    $match: {count: 5}  
  }  
]);
```

Is this a solution?

Not really. What if a reviewer has not reviewed any proposals (i.e., has no reviews array)?

The **\$size** operator raises an error.

How to fix this?

submitters collection

{sid: , name: , email: ,
proposals: [{pid: , title: , area: }, ...]}

reviewers collection

{rid: , name: , email: , expertise: ,
reviews: [{pid: , score: }, ...]}

```
{sid: , name: , email: ,
  proposals: [{pid: , title: , area: }, ...]}
```

QUESTION 1 (CONT'D)

```
{rid: , name: , email: , expertise: ,
  reviews: [{pid: , score: }, ...]}
```

- c) Construct a MQL query to find the title and average score of each proposal in the Database area. Order the result by average score descending.

```
db.submitters.aggregate([
  { $unwind: {path: '$proposals'}},
  { $match: {'proposals.area': 'Database'}}, // keep only Database proposals
  { $lookup: { // get the reviews for each proposal
    from: 'reviewers',
    localField: 'proposals.pid',
    foreignField: 'reviews.pid',
    as: 'proposalReviews'
  }},
  { $unwind: {path: '$proposalReviews'}},
  { $unwind: {path: '$proposalReviews.reviews'}},
  // keep only the reviews for the Database proposals
  { $match: { $expr: { $eq: ['$proposals.pid', '$proposalReviews.reviews.pid'] } }},
  { $group: { // calculate the average score for each proposal
    _id: {pid: '$proposals.pid', title: '$proposals.title'},
    avgScore: { $avg: '$proposalReviews.reviews.score' }
  }},
  { $project: { _id: 0, title: '$_id.title', avgScore: { $round: ['$avgScore', 2] } }},
  { $sort: { avgScore: -1 } }
]);
```

submitters collection

{sid: , name: , email: ,
proposals: [{pid: , title: , area: }, ...]}

reviewers collection

{rid: , name: , email: , expertise: ,
reviews: [{pid: , score: }, ...]}

QUESTION 1 (CONT'D)

- c) Construct a MQL query to find the title and average score of each proposal in the Database area. Order the result by average score descending.

```
db.reviewers.aggregate([
  {$unwind: {path: '$reviews'}},
  {$group: { // calculate the average score for each proposal
    _id: '$reviews.pid',
    avgScore: {$avg: '$reviews.score'}
  }},
  {$lookup: { // get the proposal information for each review
    from: 'submitters',
    localField: '_id', // Note: _id is the pid of the review
    foreignField: 'proposals.pid',
    as: 'proposalInfo'
  }},
  {$unwind: {path: '$proposalInfo'}},
  {$unwind: {path: '$proposalInfo.proposals'}},
  // match reviews with proposals and keep only the Database proposals
  {$match: {$expr: {$and: [{$eq: ['$ _id', '$proposalInfo.proposals.pid']},
    {$eq: ['$proposalInfo.proposals.area', 'Database']}]}}},
  {$project: {_id: 0, title: '$proposalInfo.proposals.title', avgScore: {$round: ['$avgScore', 2]}},
  {$sort: {avgScore: -1}}
]);
```

QUESTION 2

a) A page can hold either 3 records or 10 (search-key, pointer) index entries. If a database contains n records, then how many pages are needed to store both the data file and a single-level dense index?

- i. $n/30$
- ii. $3n/10$
- iii. $10n/3$
- iv. $13n/30$

number of records = n $bf_{\text{data file}} = 3$ $bf_{\text{index}} = 10$

Pages needed for the data file: $n/3$

Pages needed for the single-level dense index: $n/10$

Total pages needed: $n/3 + n/10 = \underline{13n/30}$

QUESTION 2 (CONTD)

- b) In a B⁺-tree, if the search-key value is 12 bytes, the page size is 1024 bytes and a pointer is 6 bytes, then the maximum number of search-key values that can be stored in each non-leaf node of the B⁺-tree is:
- i. 54
 - ii. 56
 - iii. 57
 - iv. 58

Each index entry is $12 + 6 = 18$ bytes.

Therefore, $\lfloor 1024 / 18 \rfloor = 56$ index entries (i.e., search-key values) can be stored in a non-leaf node.

QUESTION 2 (CONTD)

c) What is the minimum number of search-keys in any non-root node for a B⁺-tree in which the maximum number of search-keys in a node is 4?

a) 1

b) 2

c) 3

d) 4

The fan out of the B⁺-tree, n , is 5 (i.e., one more than the number of values).

For internal nodes, the minimum number of values is $\lceil n/2 \rceil - 1 = \lceil 5/2 \rceil - 1 = 2$.

For leaf nodes, the minimum number of values is $\lceil (n-1)/2 \rceil = \lceil 4/2 \rceil = 2$.

Therefore, for any non-root node the minimum number of values is 2.

QUESTION 3

The relation *Sailor*(sailorId, name, rating, age) is not sorted. Assume each attribute is 25 bytes, the page size is 1,000 bytes and there are 11,000 tuples. For the following questions, apply external sorting using a buffer of 11 pages.

- a) How many sorted runs will be produced in the **sorting pass (pass 0)**?
- i. 10
 - ii. 11
 - iii. 100
 - iv. 110

tuple size: 100 bytes
 bf_{Sailor} : 10 tuples/page
 B_{Sailor} : 1100 pages
buffer M : 11 pages

In the sorting pass, 11 pages at a time are sorted in memory.

Therefore, $\lceil 1100/11 \rceil = 100$ sorted runs are produced.

QUESTION 3 (CONTD)

b) What is the total number of passes required to sort the relation completely (including the sorting pass)?

i. 2

ii. 3

iii. 4

iv. 5

tuple size: 100 bytes
 bf_{Sailor} : 10 tuples/page
 B_{Sailor} : 1100 pages
buffer M : 11 pages

After the sorting pass there are 100 sorted runs.

In merge pass 1, 10 runs at a time are merged, producing 10 sorted runs.

In merge pass 2, these 10 sorted runs are merged to produce the final output.

Therefore, the total number of passes = 3.

QUESTION 3 (CONTD)

c) What is the total page I/O cost of sorting the relation?

- i. 4,400
- ii. 5,500
- iii. 6,600
- iv. 7,770

tuple size: 100 bytes
 bf_{Sailor} : 10 tuples/page
 B_{Sailor} : 1100 pages
buffer M : 11 pages

There are a total of three passes.

In each pass the whole relation (1100 pages) is read and written.

Therefore, the total page I/O cost is $3 * 2 * 1100 = \underline{6,600}$

QUESTION 4

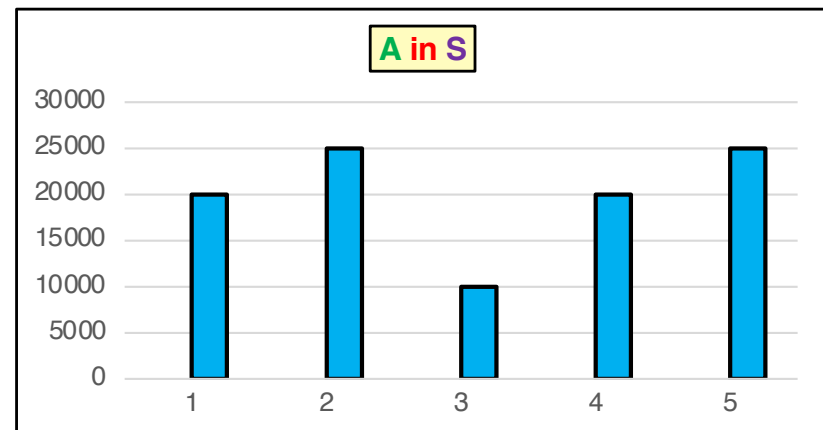
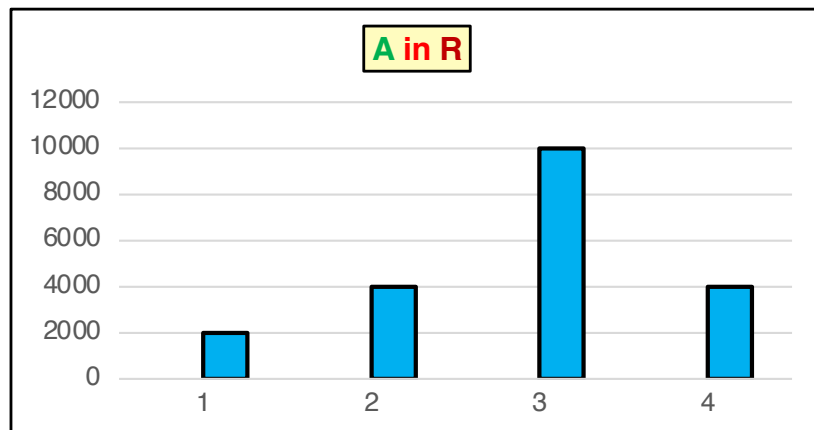
Consider the two relations $R(\underline{B}, \underline{C}, A)$ and $S(\underline{B}, A, Y)$.

R contains 20,000 tuples and S contains 100,000 tuples.

Assume that for both relations, 10 tuples fit per page (i.e., the size of R is 2,000 pages and that of S is 10,000 pages).

The possible value of attribute A in R is one of $\{1, 2, 3, 4\}$, whereas the possible value of A in S is one of $\{1, 2, 3, 4, 5\}$.

The following histograms present statistical information about the occurrence of values for A in R and S (e.g., there are 2,000 tuples with $A=1$ in R and 20,000 tuples with $A=1$ in S).



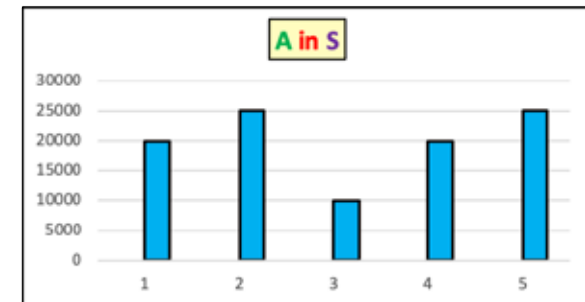
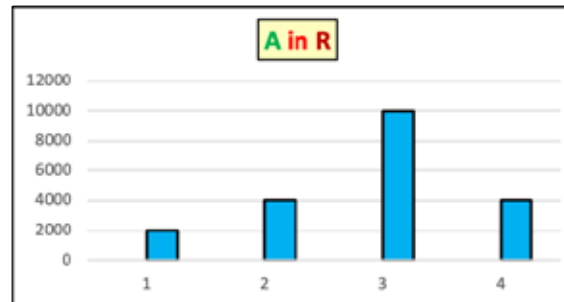
$R(\underline{B}, C, A)$
 $S(\underline{B}, A, Y)$

QUESTION 4 (CONTD)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

a) How many **tuples** are there in the query result of $(R \text{ JOIN}_{R.A=S.A} S)$ (i.e., what is the cardinality of the output)?

- i. 20,000
- ii. 100,000
- iii. 320,000,000
- iv. 400,000,000



A is not a key for R or S.

According to the histograms, each tuple of R with value $R.A=1$ (of which there are 2,000) can join with 20,000 tuples of S with $S.A=1$. That is, the join result will contain $2,000 * 20,000 = 40 * 10^6$ tuples for $R.A=S.A=1$.

Performing the same computation for A equals 2, 3, 4 and 5 the result will contain $(40 + 100 + 100 + 80 + 0) * 10^6 = \underline{320,000,000}$ tuples.

R(B, C, A)
S(B, A, Y)

QUESTION 4 (CONT'D)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

- b) What is the **minimum page I/O cost** to compute $(R \text{ JOIN}_{R.A=S.A} S)$ using block nested-loop join; **how many buffer pages** are needed?
- Minimum page I/O cost is 12,000; 2,000 buffer pages are needed.
 - Minimum page I/O cost is 12,000; 2,002 buffer pages are needed.
 - Minimum page I/O cost is 320,000 and 2,002 buffer pages are needed.
 - Minimum page I/O cost is 320,000; 12,000 buffer pages are needed.

Since there is no index, at least both relations must be read (i.e., the minimum page I/O cost is $2,000 + 10,000 = \underline{12,000}$ *independent of the algorithm*).

For block nested-loop join, the smaller relation R (2,000 pages) needs to be able to be kept in memory, so that S is scanned only once.

Since 1 page is needed for reading S and 1 page for holding the output, 2,002 buffer pages are needed.

$R(\underline{B}, C, A)$
 $S(\underline{B}, A, Y)$

QUESTION 4 (CONT'D)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

- c) We want to compute $(R \text{ JOIN}_{R.A=S.A} S)$ using block nested-loop join with R *as the outer relation*. What is the **minimum number of buffer pages needed** in order to achieve a page I/O cost of 42,000?

- i. 502
- ii. 2,002
- iii. 10,002
- iv. 12,002

Since R (2,000 pages) is the outer relation and the total page I/O cost is 42,000, S needs to be scanned $(42,000 - 2,000) / 10,000 = 4$ times.

If S is scanned 4 times, this means that 4 “blocks” of R need to be read to join with S , and each “block” must be at least $2,000 / 4 = 500$ pages.

Since, 1 page is needed for reading S and 1 page for the output, in total 502 buffer pages are needed.

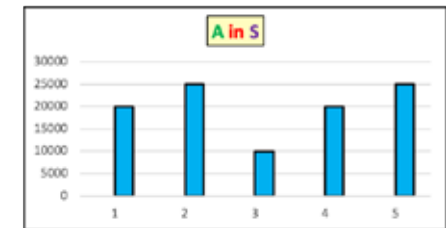
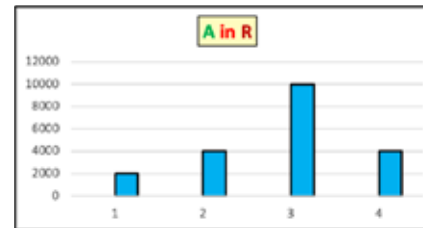
$R(\underline{B}, C, A)$
 $S(\underline{B}, A, Y)$

QUESTION 4 (CONTD)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

d) We want to compute $(R \text{ JOIN}_{R.A=S.A} S)$ using hash join with R *as the build input*. How many buckets should be used for partitioning and what is the minimum buffer requirement?

- i. 4 buckets; 202 buffer pages.
- ii. 4 buckets; 1,002 buffer pages.
- iii. 5 buckets; 202 buffer pages.
- iv. 5 buckets; 102 buffer pages.



The join condition is $R.A=S.A$ and there are only four values of A in R . Therefore, 4 buckets should be used.

According to the R histogram, 10,000 tuples of the build input A , have value $R.A=3$. Thus, the bucket size should be $\lceil 10,000 / 10 \rceil = 1,000$ pages ($bf=10$) so that each bucket of R can fit into the buffer.

In addition, we need 1 page for reading S and 1 page for the output.

Therefore, 1,002 buffer pages are needed.

$R(\underline{B}, C, A)$
 $S(\underline{B}, A, Y)$

QUESTION 4 (CONT'D)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

- e) Given that $R.B$ is a **not null** foreign key referencing $S.B$, **how many tuples** are in the query result of $(R \text{ JOIN}_{R.B=S.B} S)$?
- i. 2,000
 - ii. 20,000
 - iii. 30,000
 - iv. 120,000

Each tuple of R joins with exactly one tuple in S .

Therefore, the size of the join result is the same as the size of R , namely, 20,000 tuples.

$R(\underline{B}, C, A)$
 $S(\underline{B}, A, Y)$

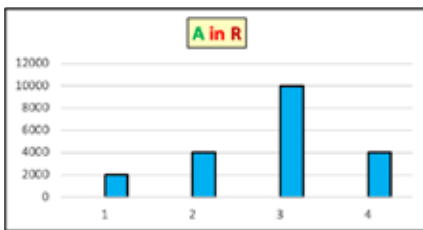
QUESTION 4 (CONT'D)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

- f) We want to compute $(R \text{ JOIN}_{R.B=S.B} S)$ using indexed nested-loop join with *R as the outer relation*. Assume that there is a hash index on $S.B$ with no overflow buckets (i.e., finding an index entry has page I/O cost 1). What is the *join page I/O cost*?
- i. 12,000
 - ii. 22,000
 - iii. 40,000
 - iv. 42,000

For each tuple of R , the hash index entry with the corresponding value of B is found and the pointer followed to the tuple of S for a page I/O cost of 2 per tuple. This is repeated for all 20,000 tuples of R , for a total page I/O cost of $20,000 * 2 = 40,000$.

In addition, the 2,000 pages of R must be read for a total page I/O cost of $40,000 + 2,000 = \underline{42,000}$.



$R(\underline{B}, \underline{C}, A)$
 $S(\underline{B}, A, Y)$

QUESTION 4 (CONT'D)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

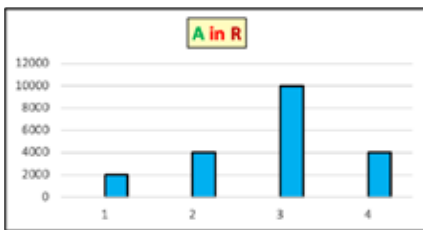
g) How many tuples are there in the result of $((\sigma_{A=1} R) \text{ JOIN}_{R.B=S.B} S)$ and what is the minimum query processing page I/O cost using indexed nested-loop join with *R as the outer relation*. Assume that the only index is a hash index on *S.B* with no overflow buckets.

- The result has 2,000 tuples; the page I/O cost is 6,000.
- The result has 2,000 tuples; the page I/O cost is 12,000.
- The result has 20,000 tuples; the page I/O cost is 40,000.
- The result has 20,000 tuples; the page I/O cost is 42,000.

According to the *A* in *R* histogram, there are only 2,000 tuples in *R* with *R.A=1*. Each of these tuples, matches exactly 1 tuple in the join with *S* (*R.B=S.B*). Thus, the result contains 2,000 tuples.

Finding the matching *S* tuple has page I/O cost 2 (see previous question) and so finding all matches for the 2,000 tuples has page I/O cost 4,000.

Adding the cost of reading *R* (2,000 page I/Os), the minimum query processing page I/O cost is 6,000.

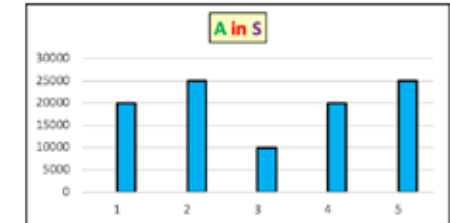


$R(\underline{B}, C, A)$
 $S(\underline{B}, A, Y)$

QUESTION 4 (CONT'D)

Relation	R	S
# tuples	20,000	100,000
# pages	2,000	10,000

h) How many tuples are there in the result of $((\sigma_{A=1}R) \text{ JOIN}_{R.B=S.B} (\sigma_{A=3}S))$ and what is the minimum query processing page I/O cost using index nested-loop join with R as the outer relation. Assume that the only index is a hash index on $S.B$ with no overflow buckets.



- The result has 200 tuples; the page I/O cost is 600.
- The result has 200 tuples; the page I/O cost is 6,000.
- The result has 2,000 tuples; the page I/O cost is 6,000.
- The result has 2,000 tuples; the page I/O cost is 12,000.

According to the A in S histogram, among the 2,000 tuples in the result of $((\sigma_{A=1}R) \text{ JOIN}_{R.B=S.B} S)$, only $10,000 / 100,000 = 0.1$ (i.e., 10%) are expected to satisfy the additional condition $S.A=3$. Thus, the result is expected to contain $2,000 * 0.1 = 200$ tuples.

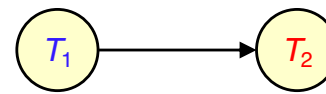
The minimum query processing page I/O cost is 6,000, the same as in the previous question.

QUESTION 5

a) Consider the following schedules of transactions T_1 and T_2 . Indicate for each schedule whether it is serial, (conflict) serializable or not serializable.

i.

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(B)

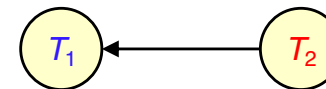


Precedence Graph

Serial: $T_1 T_2$

ii.

T_1	T_2
read(A)	
	read(A)
write(A)	
	write(B)

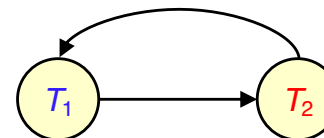


Precedence Graph

Serializable: $T_2 T_1$

iii.

T_1	T_2
read(A)	
	read(A)
	write(A)
write(A)	



Precedence Graph

Not Serializable

QUESTION 5 (CONTD)

b) Consider the schedule shown on the right.

i. Is the schedule recoverable? Why?

It is recoverable (T_2 reads data item A written by T_1 , but T_1 commits before T_2).

Is the schedule cascadeless? Why?

It is not cascadeless (T_2 reads data item A before T_1 commits).

ii. Change the time of the commits in the schedule in i. so that it becomes a cascadeless schedule.

Transaction T_2 reads the value of A written by T_1 , after T_1 commits.

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(B)
commit	
	commit

T_1	T_2
read(A)	
write(A)	
commit	
	read(A)
	write(B)
	commit

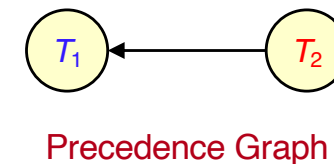
QUESTION 5 (CONTD)

- c) Modify the schedule below according to the strict 2PL protocol by adding **lock-s**, **lock-x**, **unlock** statements. Explain briefly whether the schedule is allowed by 2PL.

The schedule is serializable and so it is allowed by 2PL .

However, T_1 must wait until T_2 unlocks A.

T_1	T_2
	lock-s(A)
	read(A)
lock-s(A)	
read(A)	
lock-x(A) $\Rightarrow$ wait	
write(A)	
	lock-x(B)
	write(B)
	unlock(A)
	unlock(B)



QUESTION 5 (CONTD)

- d) Modify the following schedule according to the timestamp-ordering protocol by adding **RTS** (read timestamp) and **WTS** (write timestamp) statements. Assume that the timestamps of T_1 and T_2 are 2 and 1, respectively. The initial read and write timestamps of A and B are both 0.

T_1 [TS=2]	T_2 [TS=1]
	read(A) RTS(A)=1; WTS(A)=0
read(A) RTS(A)=2; WTS(A)=0	
write(A) RTS(A)=2; WTS(A)=2	
	write(B) RTS(B)=0; WTS(B)=1

Read

If $TS(T_i) < WTS(Q)$ **rollback**

If $TS(T_i) \geq WTS(Q)$

$RTS(Q) = \max(TS(T_i), RTS(Q))$

Write

If $TS(T_i) < RTS(Q)$ **rollback**

If $TS(T_i) < WTS(Q)$ **ignore**

Otherwise $WTS(Q) = TS(T_i)$

QUESTION 5 (CONTD)

- e) Modify the following schedule according to the multi-version timestamp-ordering protocol by adding **RTS** (read timestamp) and **WTS** (write timestamp) statements after each instruction and specify the versions of the items read and written. Assume that the timestamps of T_1 and T_2 are 1 and 2, respectively and that the initial versions of items are A_0 and B_0 with values 0 for their read and write timestamps. Complete the correct version numbers (e.g., $\text{read}(A_0)$ instead of $\text{read}(A)$).

T_1 [TS=1]	T_2 [TS=2]
	$\text{read}(A_0)$ $\text{RTS}(A_0)=2$; $\text{WTS}(A_0)=0$
$\text{read}(A_0)$ $\text{RTS}(A_0)=2$; $\text{WTS}(A_0)=0$	
$\text{write}(A)$	$\text{TS}(T_1)=1 < \text{RTS}(A_0)=2 \Rightarrow \text{rollback}$ (since $\text{TS}(T_1) < \text{RTS}(A_0)$)
	$\text{write}(B_1)$ $\text{RTS}(B_1)=2$; $\text{WTS}(B_1)=2$

Does T_2 need to be rolled back?
No

Read

Reads always succeed

set $\text{RTS}(Q_k) = \max(\text{TS}(T_i), \text{RTS}(Q_k))$

Write

If $\text{TS}(T_i) < \text{RTS}(Q_k)$ **rollback**

If $\text{TS}(T_i) = \text{WTS}(Q_k)$

overwrite contents

If $\text{TS}(T_i) > \text{WTS}(Q_k)$

create new version

set $\text{R/WTS}(Q') = \text{TS}(T_i)$